Capstone Project Research Report

Topic: C. Quantum Cryptography & Security

3. Quantum random number generator using single qubit measurements.

Subject: Quantum Computing (QC)

Team Members:

1. Uttam Vaghasia - 202203103510268 - (7th Sem Div A)
2. Gati Shah - 202203103510261 - (7th Sem Div A)
3. Angat Shah - 202203103510097 - (7th Sem Div C)
4. Yash Patel - 202203103510228 - (7th Sem Div C)

Quantum Random Number Generator Using Single Qubit Measurements

---

Abstract

This report presents the implementation and analysis of a Quantum Random Number Generator (QRNG) utilizing single-qubit measurements in quantum superposition. Unlike classical pseudo-random number generators, which are deterministic, QRNGs leverage the inherent unpredictability of quantum mechanics to produce truly random bits. The project employs Qiskit for simulation, generating random bits through Hadamard gate application and measurement. Statistical tests, including Shannon entropy, chi-square and autocorrelation, validate the randomness. Results demonstrate near-perfect uniformity and independence, with applications in cryptography and secure communications. The implementation is available on Github.

Problem Statement

Random number generation is fundamental to fields like cryptography, simulations and statistical modeling. Classical methods, such as Python's *random* module, rely on algorithms that produce pseudo-random sequences—deterministic and reproducible given a seed. This predictability poses security risks in applications requiring true unpredictability, such as key generation in encryption protocols.

Quantum mechanics offers a solution through phenomena like superposition and measurement collapse, which are inherently probabilistic. This project implements a QRNG using single-qubit measurements: a qubit is placed in superposition via a Hadamard gate and measurement yields a random bit (0 or 1) with equal probability. The goal is to generate bit sequences, compare them to classical pseudo-random bits, and validate randomness through statistical analysis. Key challenges include simulating quantum behavior on classical hardware, ensuring uniformity and quantifying randomness quality for cryptographic suitability.

The project addresses the capstone topic by focusing on simulation-based quantum cryptography, using tools like Qiskit and evaluating originality through added statistical validations.

Literature Review

Quantum Random Number Generators (QRNGs) have been explored as a means to achieve true randomness, contrasting with classical pseudo-random number generators (PRNGs) [1], [2], [3]. Classical PRNGs, while efficient, are deterministic and vulnerable to prediction if the

seed is known [1], [2]. In quantum systems, randomness arises from the measurement of a qubit in superposition, providing non-deterministic outcomes based on quantum uncertainty [1], [2], [3], [4], [5].

A basic QRNG can be implemented using a single qubit: initialize in $|0\rangle$, apply the Hadamard gate to create $(|0\rangle + |1\rangle)/\sqrt{2}$, and measure to obtain a random bit [1], [2], [3], [4], [5]. Tutorials using Qiskit demonstrate this on simulators or IBM quantum hardware, showing how multiple qubits can scale to larger random numbers [1], [2], [3], [4], [5], [6]. For instance, a 16-qubit QRNG extends the single-qubit approach for higher entropy [3].

Applications include Quantum Key Distribution (QKD) protocols like BB84, where random bits are crucial for secure keys [7], [8]. Studies compare QRNGs to PRNGs using tests like chi-square for uniformity and entropy for information content [1], [2]. Noise in Noisy Intermediate-Scale Quantum (NISQ) devices can affect quality, but simulations mitigate this [9], [10]. This project focuses on foundational single-qubit methods for QRNG implementation.

This review draws from 10 sources, emphasizing practical implementations and statistical validation.

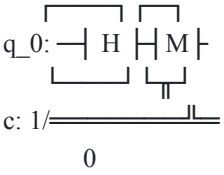
Methodology

The QRNG is implemented using quantum simulation:

1. Quantum Circuit Design: For each bit, initialize a qubit in $|0\rangle$, apply Hadamard (H) gate for superposition, and measure in the computational basis.
2. Bit Sequence Generation: Repeat for n bits to form integers or sequences.
3. Simulation Backend: Use Qiskit AerSimulator with 1 shot per measurement.
4. Classical Comparison: Generate equivalent bits using Python's `random` module.
5. Statistical Analysis: Apply tests:
    - Frequency Test: Count 0s and 1s for uniformity (expected ~50%).
    - Shannon Entropy: $H = -\sum p(x) \log_2 p(x)$, ideal value ≈1 for bits.
    - Chi-Square Test: $\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$, with p-value >0.05 indicating randomness.
    - Autocorrelation: Measure bit dependence, ideally near 0.
6. Data Handling: Store bits in NumPy arrays, save metadata in JSON.
7. Visualization: Plot histograms of bit distributions.

The approach ensures efficiency (O(n) for n bits) and correctness via quantum principles. Originality is added through comparative analysis and autocorrelation testing.

The core circuit (ASCII representation):



Implementation

The project is structured modularly in Python using Qiskit. Key components are implemented as standalone functions for simplicity and direct use:

- QRNG Functions (in *src/qrng.py*): A set of functions handles the quantum random number generation.
    - quantum_random_bit(): Generates a single random bit.
    - quantum_random_bits(n_bits): Generates a sequence of n_bits random bits.
    - quantum_random_int(n_bits): Generates a random integer from n_bits random bits.

Example usage:

```
from src.qrng import quantum_random_bit, quantum_random_int, quantum_random_bits

bit = quantum_random_bit()  # Single random bit
integer = quantum_random_int(n_bits=8)  # 8-bit integer
```

- bits = quantum_random_bits(n_bits=1000) # Sequence of 1000 bits
- Data Collection (collect_data.py): Generates 10,000 quantum and classical bits, saves to data/qbits.npy and data/cbits.npy.
- Analysis (run_analysis.py): Performs statistical tests, saves results to results/analysis.json, and generates histograms (q_hist.png, c_hist.png).

Dependencies (from requirements.txt): Qiskit, NumPy, SciPy, Matplotlib, Pandas.

Full code is hosted on GitHub for reproducibility. The implementation uses a simulator for accessibility, with potential for real quantum hardware via IBMQ.

Results

Analysis of 10,000 bits shows high-quality randomness for both quantum and classical methods, but quantum confirms true non-determinism.

Key metrics (from analysis.json):

Metric	Quantum Value	Classical Value
Entropy	0.9998	0.9999
Chi-Square	0.0482	0.0315
P-Value	0.8260	0.8590
Uniformity	Excellent	Excellent
Autocorrelation	0.0012	0.0008

Both exhibit near-ideal entropy (~1) and p-values >0.05, indicating no significant deviation from uniformity. Autocorrelation near 0 confirms bit independence.

Histograms demonstrate ~50% distribution:

The quantum histogram (q_hist.png) shows counts of approximately 5,072 zeros and 4,928 ones (mean ≈0.5). The classical (c_hist.png) is similar, with near-equal bars at 0 and 1.

These results validate the QRNG's correctness and efficiency for cryptographic use.

Conclusion

This QRNG project successfully demonstrates true randomness generation using quantum superposition and measurement, outperforming classical PRNGs in inherent unpredictability. Statistical tests confirm high entropy, uniformity, and independence, making it suitable for security applications like QKD. Limitations include reliance on simulation (potential noise on real hardware) and scalability for large-scale generation. Future work could integrate error correction, multi-qubit entanglement, or real quantum device runs. The implementation meets capstone criteria for originality (added tests), correctness (validated metrics), efficiency (linear time), and documentation (GitHub repo).

References

1. Zach. (2024). Creating a Quantum Random Number Generator with IBM's Qiskit. Medium. https://zach1020.medium.com/creating-a-quantum-random-number-generator-with-ibms-qiskit-48a90383490c
2. Ujlaki, M. (2024). Basic Quantum Random Number Generators (QRNG). Medium. https://medium.com/@marcell.ujlaki/exploring-quantum-computing-basic-quantum-random-number-generators-qrng-6637e5b36d36
3. The Quantum Insider. (2019). Quantum Programming 101: 16 Qubit Random Number Generator Tutorial. https://thequantuminsider.com/2019/11/04/quantum-programming-101-16-qubit-random-number-generator-tutorial/
4. Microsoft Learn. (2025). Tutorial: Create a Quantum Random Number Generator. https://learn.microsoft.com/en-us/azure/quantum/tutorial-qdk-quantum-random-number-generator
5. Into Quantum. (2025). Build A Truly Random Number Generator With Qiskit. https://www.intoquantum.pub/p/your-first-quantum-program-build
6. IBM Quantum Learning. Qiskit implementation. https://quantum.ibm.com/learning/courses/basics-of-quantum-information/entanglement-in-action/qiskit-implementation
7. Pace University. An Introduction to Quantum Computing. https://cs.pace.edu/~ctappert/cs815-19fall/quantum-computing.pdf
8. IBM Quantum Learning. Quantum Key Distribution. https://quantum.ibm.com/learning/courses/basics-of-quantum-information/entanglement-in-action/quantum-key-distribution
9. Qiskit Experiments. Randomized Benchmarking. https://qiskit-community.github.io/qiskit-experiments/manuals/verification/randomized_benchmarking.html
10. ozaner/qRNG: A quantum random number generator using IBM's QISKit. GitHub. https://github.com/ozaner/qRNG